

1.inheritance

```
class Dog extends Animal {  
  |  override def bark: String = "Woof"  
  |}  
class Dog
```

```
scala> object Main extends App {  
  |  val d = new Dog  
  |  println(d.bark) // prints: Woof  
  |}  
object Main
```

```
scala> val a: Animal = new Dog  
val a: Animal = Dog@754e3050
```

```
scala>  
a.bark  
val res14: String = Woof
```

2.Primary

```
object Test {  
|  def main(args: Array[String]): Unit = {  
|  
|  
|  val d = new Duck(5, 2)  
|  
|  
|  println("Size: " + d.size)  
|  println("Age: " + d.age)  
|  
|  }  
|}  
object Test
```

3.method override

```
class Animal {  
  def sound() = {  
    println("Animals make sound")  
  }  
}
```

```
class Dog extends Animal {  
  override def sound() = {  
    println("Dog barks")  
  }  
}
```

4. Abstract Class :

```
abstract class Emp(val name: String) {
  def id: Int
  def work(): Unit
  def displayInfo(): Unit = {
    println(s"Employee Name: $name, Employee ID: $id")
  }
}

class Manager(name: String, val id: Int) extends Emp(name) {
  def work(): Unit = {
    println(s"Manager $name is managing the team.")
  }
}

class Developer(name: String, val id: Int) extends Emp(name) {
  def work(): Unit = {
    println(s"Developer $name is writing code.")
  }
}

object Main extends App {
  val mgr = new Manager("Alice", 101)
  val dev = new Developer("Bob", 202)
  mgr.displayInfo()
  mgr.work()
  dev.displayInfo()
  dev.work()
}
```

5. SUPER CLASS CONSTRUCTOR:

```
class Person(name: String, dept: String) {
  def display(): Unit = {
    println("Name: " + name)
    println("Department:" + dept)
  }
}
```